Variance Reduction for Training Neural Bayes Estimators

Kai Marns-Morris

Honours (BPhil) 2026

The University of Western Australia

Supervisors: Dr Michael Bertolacci & A/Prof Edward Cripps

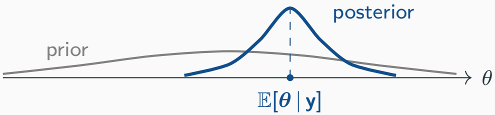
Background: neural Bayes estimators

Bayesian inference

- Fix a model: parameters $\boldsymbol{\theta}$ with prior $p(\boldsymbol{\theta})$, and data $\mathbf{y}$ with likelihood $p(\mathbf{y} \mid \boldsymbol{\theta})$.
- Seeing $\mathbf{y}$, Bayes' rule updates the prior into the **posterior distribution**:

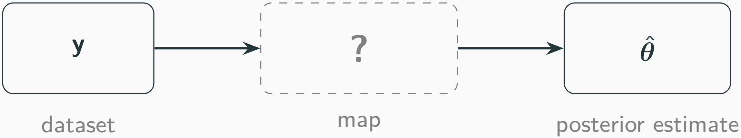
$$p(\boldsymbol{\theta} \mid \mathbf{y}) \;\propto\; p(\mathbf{y} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta}).$$

- The posterior is our full, updated belief about $\boldsymbol{\theta}$.

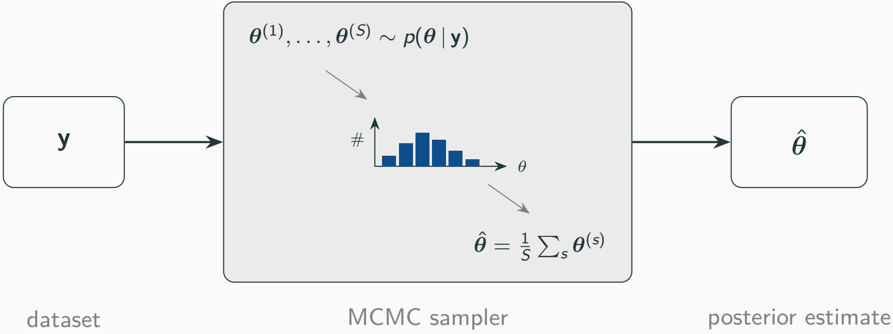


Point estimation

- Often we do not want the whole posterior — just a single **point estimate** $\hat{\theta}$ of θ.
- The usual choice is the **posterior mean** $\hat{\theta} = \mathbb{E}[\theta \mid \mathbf{y}]$.
- But it is rarely available in closed form — so we need a **procedure** that turns $\mathbf{y}$ into $\hat{\theta}$:

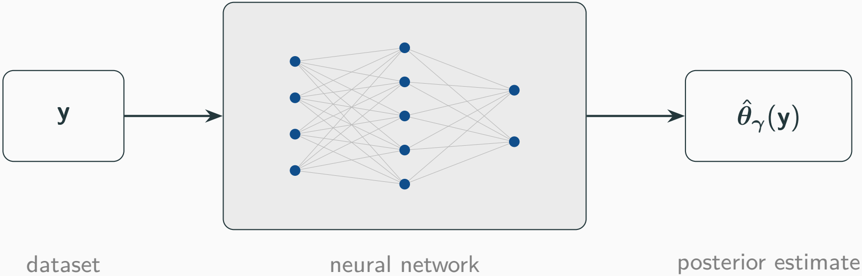


Method (1): Markov chain Monte Carlo



- Accurate and general — but must be re-run from scratch for every new $\mathbf{y}$, and can be slow.
- Do inference for *many* datasets and you pay that cost *every time*.

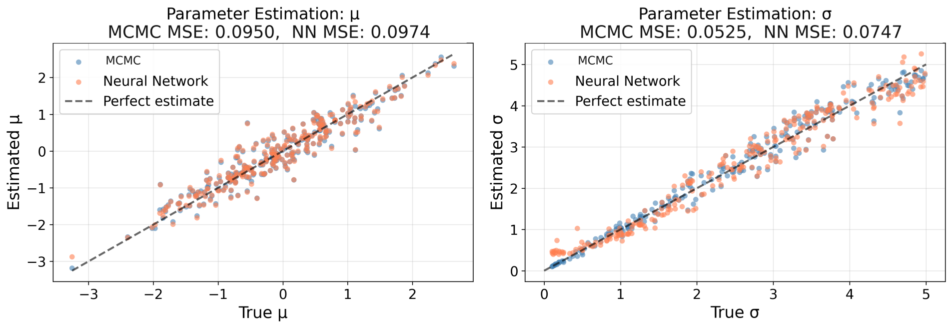
Method (2): Neural Bayes estimator



- A **neural network** $\hat{\boldsymbol{\theta}}_\gamma(\cdot)$, trained *once* — a *neural Bayes estimator*.
- Training data is free: **simulate** pairs $(\boldsymbol{\theta}, \mathbf{y})$ and adjust γ so $\hat{\boldsymbol{\theta}}_\gamma(\mathbf{y}) \approx \boldsymbol{\theta}$.
- Then each new dataset is a single **forward pass** — the cost is amortised.

Does it work?

A simple test: $y_1, \ldots, y_n \sim \mathcal{N}(\mu, \sigma^2)$ with $\mu \sim \mathcal{N}(0, 1)$ and $\sigma \sim \mathrm{Uniform}(0, 5)$.



The network (orange) matches MCMC (blue): it has learned the posterior-mean map. *Why*? — the loss it was trained with.

The loss picks which summary you estimate

The network minimises the **Bayes risk** $L(\gamma) = \mathbb{E}_{\boldsymbol{\theta},\mathbf{y}}\big[\ell(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}_\gamma(\mathbf{y}))\big]$ — and the loss ℓ decides *which* posterior summary it targets:

loss $\ell(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}})$	best estimate (its minimiser)
squared $\lVert\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}\rVert^2$	posterior **mean** $\mathbb{E}[\boldsymbol{\theta} \mid \mathbf{y}]$
absolute $\lvert\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}\rvert$	posterior **median**
quantile $(\alpha - \mathbf{1}_{\{\boldsymbol{\theta} < \hat{\boldsymbol{\theta}}\}})(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$	posterior **quantile**

Bayes risk is not available in closed form, so we *estimate it by Monte Carlo*:

$$L(\gamma) \;\approx\; \tfrac{1}{N}\sum_{i=1}^{N} \ell\big(\boldsymbol{\theta}_i, \hat{\boldsymbol{\theta}}_\gamma(\mathbf{y}_i)\big), \quad (\boldsymbol{\theta}_i, \mathbf{y}_i) \sim \text{model}.$$

The idea

In training we estimate the risk by **Monte Carlo**:

$$L(\gamma) \;\approx\; \frac{1}{N}\sum_{i=1}^{N} \ell\big(\boldsymbol{\theta}_i, \hat{\boldsymbol{\theta}}_\gamma(\mathbf{y}_i)\big), \qquad (\boldsymbol{\theta}_i, \mathbf{y}_i) \sim \text{model.}$$

- So the **gradient** $g(\gamma) = \nabla_\gamma L(\gamma)$ that drives training is a random quantity with variance.
- That variance is about *how we sample the loss*, not the inference problem — so we can reduce it.

Idea 1: Rao–Blackwellisation

Analytical conditional posteriors

- Split the parameters $\boldsymbol{\theta} = (\boldsymbol{\theta}_1, \boldsymbol{\theta}_2)$. For many models, *conditioning* on $\boldsymbol{\theta}_2$ leaves the posterior of $\boldsymbol{\theta}_1$ in **closed form**:

$$p(\boldsymbol{\theta}_1 \mid \boldsymbol{\theta}_2, \mathbf{y}) \text{ known} \quad \Longrightarrow \quad \mathbb{E}[\boldsymbol{\theta}_1 \mid \boldsymbol{\theta}_2, \mathbf{y}] \text{ is analytic.}$$

- This is **conditional conjugacy** — the same structure that makes *Gibbs sampling* possible.
- So: don't sample $\boldsymbol{\theta}_1$ — integrate it out using its exact conditional mean.

Example — Bayesian linear regression $\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}$: given the noise variance σ^2, the coefficients $\boldsymbol{\beta}$ are Gaussian, so $\mathbb{E}[\boldsymbol{\beta} \mid \sigma^2, \mathbf{y}]$ is closed-form $(\boldsymbol{\theta}_1 = \boldsymbol{\beta},\ \boldsymbol{\theta}_2 = \sigma^2)$.

Expanding the Bayes risk

Under squared error, expand the risk by conditioning on $(\boldsymbol{\theta}_2, \mathbf{y})$ ($\hat{\boldsymbol{\theta}}_1 =$ the network's estimate of $\boldsymbol{\theta}_1$):

$$\mathbb{E}\left[\|\boldsymbol{\theta}_1 - \hat{\boldsymbol{\theta}}_1\|^2 \,\middle|\, \boldsymbol{\theta}_2, \mathbf{y}\right] = \underbrace{\left\|\hat{\boldsymbol{\theta}}_1 - \mathbb{E}[\boldsymbol{\theta}_1 \mid \boldsymbol{\theta}_2, \mathbf{y}]\right\|^2}_{\text{Rao–Blackwellised loss}} + \underbrace{\mathrm{Var}(\boldsymbol{\theta}_1 \mid \boldsymbol{\theta}_2, \mathbf{y})}_{\text{constant in } \gamma}.$$

- The target for $\boldsymbol{\theta}_1$ becomes its **conditional posterior mean** $\mathbb{E}[\boldsymbol{\theta}_1 \mid \boldsymbol{\theta}_2, \mathbf{y}]$ — computed exactly, not sampled.
- The residual is free of γ: averaging over $(\boldsymbol{\theta}_2, \mathbf{y})$, it is a constant that drops out of the gradient.
- So $\boldsymbol{\theta}_1$ is integrated out *analytically* instead of sampled.

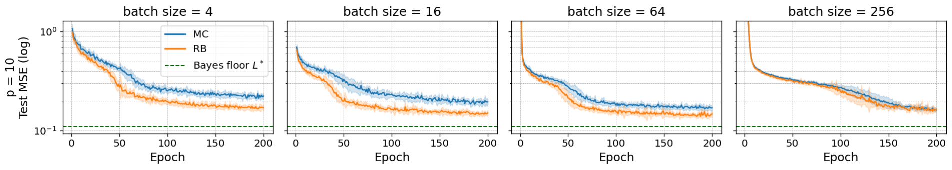
The gradient variance can only shrink

The Rao–Blackwellised gradient is the standard one **conditioned** on $\boldsymbol{\theta}_2$: $g_{\mathrm{RB}} = \mathbb{E}[g \mid \boldsymbol{\theta}_2, \mathbf{y}]$. The law of total variance gives

$$\mathrm{Var}(g) \;=\; \underbrace{\mathrm{Var}\left(\mathbb{E}[g \mid \boldsymbol{\theta}_2, \mathbf{y}]\right)}_{\mathrm{Var}(g_{\mathrm{RB}})} + \underbrace{\mathbb{E}\left[\,\mathrm{Var}(g \mid \boldsymbol{\theta}_2, \mathbf{y})\right]}_{\succeq 0}.$$

- Hence $\mathrm{Var}(g_{\mathrm{RB}}) \preceq \mathrm{Var}(g)$ — never larger, for any model or architecture.
- For a linear network the reduction reaches the fitted weights too.

Does it pay off? Bayesian linear regression



$p = 10$ coefficients; curves are mean $\pm$ 1 s.d. over repeated training runs.

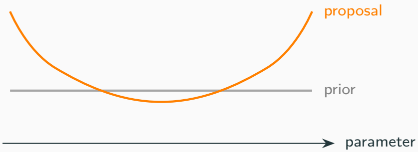
- Integrate out the coefficients β; a Gibbs sampler gives the Bayes-optimal floor L^* (green).
- RB reaches a **lower test error** at the small and moderate batches — where the training gradient is noisiest.
- The advantage **shrinks as the batch grows**: by batch 256 the curves meet.

Idea 2: importance sampling

A more general lever

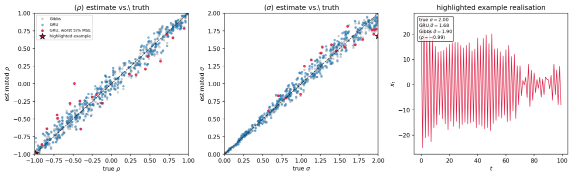
No conjugacy required — unlike Rao–Blackwell.

- Simulate from a **proposal** that oversamples the informative regions, then **reweight** back to the original risk (unbiased for any proposal).
- But no guarantee: bad weights can *increase* variance and shrink the effective sample size.

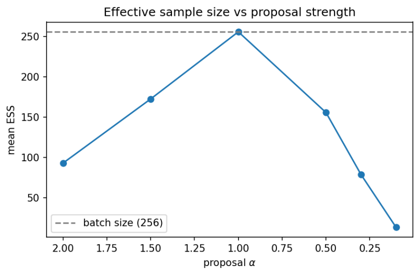


a proposal oversampling the hard edges, then reweighted

Importance sampling: the result



On an AR(1) model, the GRU already tracks the Gibbs (Bayes) reference across the prior — **little error left to recover**.



Push the proposal hard and the effective sample size **collapses** — only adding noise.

Why it fails here: the estimator already sits near the Bayes floor, and the little error that remains lives at the $|\rho| \to 1$ boundary — which the proposals do not target.

Conclusion

Conclusion

Training a neural Bayes estimator is itself Monte Carlo — so its training variance can be reduced.

- **Rao–Blackwellisation** (needs a conjugate block): a *provable* reduction — closes $\sim$half the gap to the Bayes floor and matches the standard estimator at $\frac{1}{4}$ the batch size.
- **Importance sampling** (needs nothing special): more general, but unguaranteed and problem-dependent — here it did not help.

Conjugacy — the classical trick behind Gibbs sampling — is just as useful for *training* the estimators meant to replace it.

Thank you